

[CS209A] Tutorial 1 - JVM

Author: Yao Zhao

Part 1: Compile and Run Java Program

1. Write a simple Java program `Example1.java`:

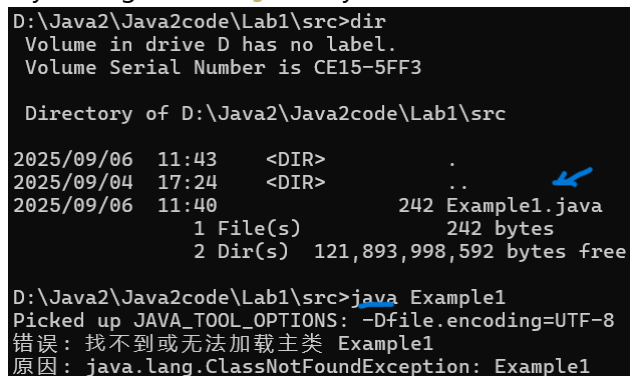
```
public class Example1 {  
    static int fib(int n) {  
        if (n <= 1)  
            return n;  
        return fib(n - 1) + fib(n - 2);  
    }  
  
    public static void main(String args[]) {  
        System.out.println(fib(10));  
    }  
}
```

2. Compile the program:

```
javac Example1.java
```

Explanation: The `javac` command compiles the `.java` source file into a `.class` bytecode file that can be executed by the Java Virtual Machine (JVM).

- If you forget to run `javac`, you will see an error: `Could not find or load main class`.



The screenshot shows a Windows command prompt window. The first part shows the directory listing for `D:\Java2\Java2code\Lab1\src`, which includes `Example1.java`. A blue arrow points to the `..` directory entry. The second part shows the command `java Example1` being executed, which results in a `ClassNotFoundException` error: `错误: 找不到或无法加载主类 Example1` and `原因: java.lang.ClassNotFoundException: Example1`.

```
D:\Java2\Java2code\Lab1\src>dir  
Volume in drive D has no label.  
Volume Serial Number is CE15-5FF3  
  
Directory of D:\Java2\Java2code\Lab1\src  
  
2025/09/06  11:43    <DIR>          .  
2025/09/04  17:24    <DIR>          ..  
2025/09/06  11:40                242 Example1.java  
               1 File(s)                242 bytes  
               2 Dir(s)  121,893,998,592 bytes free  
  
D:\Java2\Java2code\Lab1\src>java Example1  
Picked up JAVA_TOOL_OPTIONS: -Dfile.encoding=UTF-8  
错误: 找不到或无法加载主类 Example1  
原因: java.lang.ClassNotFoundException: Example1
```

3. Run the program:

```
java Example1
```

```
D:\Java2\Java2code\Lab1\src>javac Example1.java
Picked up JAVA_TOOL_OPTIONS: -Dfile.encoding=UTF-8

D:\Java2\Java2code\Lab1\src>dir
Volume in drive D has no label.
Volume Serial Number is CE15-5FF3

Directory of D:\Java2\Java2code\Lab1\src

2025/09/06  11:48    <DIR>        .
2025/09/04  17:24    <DIR>        ..
2025/09/06  11:48                505 Example1.class
2025/09/06  11:40                242 Example1.java
                2 File(s)              747 bytes
                2 Dir(s)  121,894,002,688 bytes free

D:\Java2\Java2code\Lab1\src>java Example1
Picked up JAVA_TOOL_OPTIONS: -Dfile.encoding=UTF-8
55
```

Explanation: The `java` command starts the JVM and executes the bytecode in the compiled `.class` file. The JVM handles memory management, garbage collection, and provides the runtime environment for your Java program.

4. View the class loading process:

```
java -verbose:class Example1
```

```
D:\Java2\Java2code\Lab1\src>java -verbose:class Example1
Picked up JAVA_TOOL_OPTIONS: -Dfile.encoding=UTF-8
[0.022s][info][class,load] java.lang.Object source: shared objects file
[0.023s][info][class,load] java.io.Serializable source: shared objects file
[0.024s][info][class,load] java.lang.Comparable source: shared objects file
[0.024s][info][class,load] java.lang.CharSequence source: shared objects file
[0.025s][info][class,load] java.lang.constant.Constable source: shared objects file
[0.026s][info][class,load] java.lang.constant.ConstantDesc source: shared objects file
[0.026s][info][class,load] java.lang.String source: shared objects file
[0.027s][info][class,load] java.lang.reflect.AnnotatedElement source: shared objects file
[0.027s][info][class,load] java.lang.reflect.GenericDeclaration source: shared objects file
[0.028s][info][class,load] java.lang.reflect.Type source: shared objects file
[0.028s][info][class,load] java.lang.invoke.TypeDescriptor source: shared objects file
[0.028s][info][class,load] java.lang.invoke.TypeDescriptor$OffsetField source: shared objects file
[0.029s][info][class,load] java.lang.Class source: shared objects file
[0.029s][info][class,load] java.lang.Cloneable source: shared objects file
[0.029s][info][class,load] java.lang.ClassLoader source: shared objects file
[0.029s][info][class,load] java.lang.System source: shared objects file
[0.030s][info][class,load] java.lang.Throwable source: shared objects file
[0.030s][info][class,load] java.lang.Error source: shared objects file
[0.030s][info][class,load] java.lang.ThreadDeath source: shared objects file
[0.030s][info][class,load] java.lang.Exception source: shared objects file
[0.030s][info][class,load] java.lang.RuntimeException source: shared objects file
[0.030s][info][class,load] java.lang.SecurityManager source: shared objects file
[0.031s][info][class,load] java.security.ProtectionDomain source: shared objects file
[0.031s][info][class,load] java.security.AccessControlContext source: shared objects file
[0.031s][info][class,load] java.security.AccessController source: shared objects file
[0.031s][info][class,load] java.security.SecureClassLoader source: shared objects file
[0.031s][info][class,load] java.lang.ReflectiveOperationException source: shared objects file
```

Explanation: The `-verbose:class` option prints every class the JVM loads at startup and during execution. This helps you understand:

- The sequence of class loading
- Which classes are loaded from [shared objects \(CDS archive\)](#), which JVM uses to speed up startup by preloading common classes.
- The extensive infrastructure required before your `main()` method can execute
- The total number of classes loaded for even a simple program

The simple Fibonacci program may only mention a couple of classes, but Java programs always rely on a lot of core runtime classes. For example:

- `java.lang.Object`: the root class of everything in Java.
- `java.io.Serializable`: interface that many core classes implement.
- `java.lang.Comparable`, `java.lang.CharSequence`: interfaces related to String.
- `java.lang.String`: needed for string operations.
- `java.lang.reflect.*`: used internally by the JVM, even if you don't write reflection code.

So the JVM starts by loading a bunch of fundamental classes it knows it will need. After that, it will load your own class `Example1` and any others you reference.

5. Additional Note: Source-file Execution in Java 11+

Starting from **Java 11**, the `java` command supports running a **single source file directly** (source-file mode). For example:

```
java Example1.java
```

This automatically performs two steps:

- In the background, it invokes `javac` to compile the source into a `.class` file in memory (not written to disk).
- The JVM immediately executes the compiled bytecode.

So even if you don't see a `.class` file on disk, the program can still run successfully.

```
D:\Java2\Java2code\Lab1\src>dir
Volume in drive D has no label.
Volume Serial Number is CE15-5FF3

Directory of D:\Java2\Java2code\Lab1\src

2025/09/06  11:50    <DIR>          .
2025/09/04  17:24    <DIR>          ..
2025/09/06  11:40                242 Example1.java
               1 File(s)                242 bytes
               2 Dir(s) 121,894,002,688 bytes free

D:\Java2\Java2code\Lab1\src>java Example1.java
Picked up JAVA_TOOL_OPTIONS: -Dfile.encoding=UTF-8
55
```

Limitations: This only works for single-file programs and doesn't replace the traditional compile-then-run approach for multi-file projects.

Part 2: Explore Bytecode

Understanding Java Bytecode

Java bytecode is an intermediate representation of your Java code that the JVM executes. It's platform-independent and designed for efficient execution by the JVM.

Compile and use `javap` to inspect bytecode:

```
javac Example1.java  
javap -c Example1.class  
javap -c -verbose Example1.class
```

```

PS D:\Java2\Java2code\Lab1\src> javap -c Example1.class
Picked up JAVA_TOOL_OPTIONS: -Dfile.encoding=UTF-8
Compiled from "Example1.java"
public class Example1 {
    public Example1();
        Code:
            0: aload_0
            1: invokespecial #1                  // Method java/lang/Object."<init>":()V
            4: return

    static int fib(int);
        Code:
            0: iload_0
            1: iconst_1
            2: if_icmpgt     7
            5: iload_0
            6: ireturn
            7: iload_0
            8: iconst_1
            9: isub
           10: invokestatic #7                  // Method fib:(I)I
           13: iload_0
           14: iconst_2
           15: isub
           16: invokestatic #7                  // Method fib:(I)I
           19: iadd
           20: ireturn

    public static void main(java.lang.String[]);
        Code:
            0: getstatic     #13                // Field java/lang/System.out:Ljava/io/PrintStream;
            3: bipush        10
            5: invokestatic #7                  // Method fib:(I)I
            8: invokevirtual #19               // Method java/io/PrintStream.println:(I)V
           11: return
}

```

```

PS D:\Java2\Java2code\Lab1\src> javap -c -verbose Example1.class
Picked up JAVA_TOOL_OPTIONS: -Dfile.encoding=UTF-8
Classfile /D:/Java2/Java2code/Lab1/src/Example1.class
  Last modified 2025-09-07; size 505 bytes
s
SHA-256 checksum 63d7628567f5b6127bf2955d5ec3d915b282c68acbc3b1dca198f34fdad126f4
Compiled from "Example1.java"
public class Example1
  minor version: 0
  major version: 61
  flags: (0x0021) ACC_PUBLIC, ACC_SUPER
  this_class: #8                  // Example1
  super_class: #2                 // java/lang/Object
  interfaces: 0, fields: 0, methods: 3, attributes: 1
Constant pool:
#1 = Methodref                  #2.#3      // java/lang/Object."<init>":()V
#2 = Class                      #4         // java/lang/Object
#3 = NameAndType                #5:#6      // "<init>":()V
#4 = Utf8                       java/lang/Object
#5 = Utf8                       <init>
#6 = Utf8                       ()V
#7 = Methodref                  #8.#9      // Example1.fib:(I)I
#8 = Class                      #10        // Example1
#9 = NameAndType                #11:#12    // fib:(I)I
#10 = Utf8                      Example1
#11 = Utf8                      fib
#12 = Utf8                      (I)I
#13 = Fieldref                  #14.#15    // java/lang/System.out:Ljava/io/PrintStream;
#14 = Class                      #16        // java/lang/System
#15 = NameAndType                #17:#18    // out:Ljava/io/PrintStream;
#16 = Utf8                      java/lang/System
#17 = Utf8                      out
#18 = Utf8                      Ljava/io/PrintStream;
#19 = Methodref                  #20.#21    // java/io/PrintStream.println:(I)V
#20 = Class                      #22        // java/io/PrintStream
#21 = NameAndType                #23:#24    // println:(I)V
#22 = Utf8                      java/io/PrintStream
#23 = Utf8                      println
#24 = Utf8                      (I)V
#25 = Utf8                      Code
#26 = Utf8                      LineNumberTable
#27 = Utf8                      StackMapTable
#28 = Utf8                      main
#29 = Utf8                      ([Ljava/lang/String;)V

```

```

#30 = Utf8      SourceFile
#31 = Utf8      Example1.java

{
  public Example1();
  descriptor: ()V
  flags: (0x0001) ACC_PUBLIC
  Code:
    stack=1, locals=1, args_size=1
    0: aload_0
    1: invokespecial #1          // Method java/lang/Object.<init>:()V
    4: return
 LineNumberTable:
    line 1: 0

  static int fib(int);
  descriptor: (I)I
  flags: (0x0008) ACC_STATIC
  Code:
    stack=3, locals=1, args_size=1
    0: iload_0
    1: iconst_1
    2: if_icmpgt     7
    5: iload_0
    6: ireturn
    7: iload_0
    8: iconst_1
    9: isub
   10: invokestatic  #7          // Method fib:(I)I
   13: iload_0
   14: iconst_2
   15: isub
   16: invokestatic  #7          // Method fib:(I)I
   19: iadd
   20: ireturn
 LineNumberTable:
    line 3: 0
    line 4: 5
    line 5: 7
  StackMapTable: number_of_entries = 1
    frame_type = 7 /* same */

  public static void main(java.lang.String[]);
  descriptor: ([Ljava/lang/String;)V
  flags: (0x0009) ACC_PUBLIC, ACC_STATIC
  Code:
    stack=2, locals=1, args_size=1
    0: getstatic     #13         // Field java/lang/System.out:Ljava/io/PrintStream;
    3: bipush       10
    5: invokestatic  #7          // Method fib:(I)I
    8: invokevirtual #19         // Method java/io/PrintStream.println:(I)V
   11: return
 LineNumberTable:
    line 9: 0
    line 10: 11
}
SourceFile: "Example1.java"
PS D:\Java2\Java2code\Lab1\src> |

```

Explanation: `javap` is a disassembler tool included with the JDK that reads `.class` files and shows the bytecode.

- `-c` displays the bytecode instructions for methods.
- `-verbose` shows additional details such as:
 - **Constant Pool:** A "library" of all constants, method references, and field references used by the class
 - **Method descriptors:** Type signatures showing parameter and return types
 - **Class metadata:** Version information, access flags, and inheritance
 - **LineNumberTable:** Mapping between bytecode and source code lines for debugging

Part 3: JVM Memory and Garbage Collection

1. Write a memory-intensive Java program `Example2.java`:

```
import java.util.ArrayList;
import java.util.List;

public class Example2 {
    public static void main(String[] args) {
        List<int[]> list = new ArrayList<>();
        for (int i = 0; i < 10000; i++) {
            int[] data = new int[1000]; // Allocate 4KB per array (1000 ints x 4
bytes)
            list.add(data);
        }
        System.out.println("Done");
    }
}
```

2. Compile and run with GC details:

```
javac Example2.java
java -XX:+PrintGCDetails Example2
```

```
PS D:\Java2\Java2code\Lab1\src> java -XX:+PrintGCDetails Example2
Picked up JAVA_TOOL_OPTIONS: -Dfile.encoding=UTF-8
[0.005s][warning][gc] -XX:+PrintGCDetails is deprecated. Will use -Xlog:gc* instead.
[0.011s][info] ][gc] Using G1
[0.014s][info] ][gc,init] Version: 17.0.12+8-LTS-286 (release)
[0.014s][info] ][gc,init] CPUs: 12 total, 12 available
[0.015s][info] ][gc,init] Memory: 16139M
[0.015s][info] ][gc,init] Large Page Support: Disabled
[0.016s][info] ][gc,init] NUMA Support: Disabled
[0.016s][info] ][gc,init] Compressed Oops: Enabled (Zero based)
[0.017s][info] ][gc,init] Heap Region Size: 2M
[0.017s][info] ][gc,init] Heap Min Capacity: 8M
[0.017s][info] ][gc,init] Heap Initial Capacity: 254M
[0.017s][info] ][gc,init] Heap Max Capacity: 4036M
[0.017s][info] ][gc,init] Pre-touch: Disabled
[0.017s][info] ][gc,init] Parallel Workers: 10
[0.018s][info] ][gc,init] Concurrent Workers: 3
[0.018s][info] ][gc,init] Concurrent Refinement Workers: 10
[0.018s][info] ][gc,init] Periodic GC: Disabled
[0.026s][info] ][gc,metaspace] CDS archive(s) mapped at: [0x00000248a0000000-0x00000248a0bd0000-0x000
00248a0bd0000], size 12386304, SharedBaseAddress: 0x00000248a0000000, ArchiveRelocationMode: 1.
[0.026s][info] ][gc,metaspace] Compressed class space mapped at: 0x00000248a1000000-0x00000248e10000
0, reserved size: 1073741824
[0.027s][info] ][gc,metaspace] Narrow klass base: 0x00000248a0000000, Narrow klass shift: 0, Narrow k
lass range: 0x100000000
[0.071s][info] ][gc,start] GC(0) Pause Young (Normal) (G1 Evacuation Pause)
[0.072s][info] ][gc,task] GC(0) Using 6 workers of 10 for evacuation
[0.081s][info] ][gc,phases] GC(0) Pre Evacuate Collection Set: 0.2ms
[0.082s][info] ][gc,phases] GC(0) Merge Heap Roots: 0.2ms
[0.082s][info] ][gc,phases] GC(0) Evacuate Collection Set: 6.5ms
[0.083s][info] ][gc,phases] GC(0) Post Evacuate Collection Set: 0.9ms
[0.083s][info] ][gc,phases] GC(0) Other: 1.9ms
[0.083s][info] ][gc,heap] GC(0) Eden regions: 11->0(6)
[0.083s][info] ][gc,heap] GC(0) Survivor regions: 0->2(2)
[0.084s][info] ][gc,heap] GC(0) Old regions: 0->9
```

Explanation: When your program started, the JVM set up memory management using the G1 Garbage Collector. G1 is the default collector in modern Java — its job is to automatically clean up unused objects in memory so your program doesn't run out of space. It's designed to do this efficiently and with short pauses, which helps programs run smoothly.

Shortly after startup, the JVM performed its first garbage collection event. A garbage collection event is simply a moment when the JVM pauses briefly to reclaim memory that is no longer needed. In your output, this was labeled as GC(0). The pause only lasted a few milliseconds, which is very fast and normal.

The `-XX:+PrintGCDetails` option prints detailed garbage collection information, including:

- When GC occurs (timing and frequency)
- Memory usage changes before and after GC
- GC type and duration
- Heap region utilization (Eden, Survivor, Old regions) (discuss later).

3. Limit heap size to trigger GC more frequently:

```
java -Xmx64m -XX:+PrintGCDetails Example2
```

```
PS D:\Java2\Java2code\Lab1\src> java -Xmx64m -XX:+PrintGCDetails Example2
Picked up JAVA_TOOL_OPTIONS: -Dfile.encoding=UTF-8
[0.005s][warning][gc] -XX:+PrintGCDetails is deprecated. Will use -Xlog:gc* instead.
[0.012s][info]    [gc] Using G1
[0.013s][info]    [gc,init] Version: 17.0.12+8-LTS-286 (release)
[0.014s][info]    [gc,init] CPUs: 12 total, 12 available
[0.014s][info]    [gc,init] Memory: 16139M
[0.015s][info]    [gc,init] Large Page Support: Disabled
[0.015s][info]    [gc,init] NUMA Support: Disabled
[0.015s][info]    [gc,init] Compressed Oops: Enabled (32-bit)
[0.015s][info]    [gc,init] Heap Region Size: 1M
[0.016s][info]    [gc,init] Heap Min Capacity: 8M
[0.016s][info]    [gc,init] Heap Initial Capacity: 64M
[0.016s][info]    [gc,init] Heap Max Capacity: 64M
[0.016s][info]    [gc,init] Pre-touch: Disabled
[0.016s][info]    [gc,init] Parallel Workers: 10
[0.017s][info]    [gc,init] Concurrent Workers: 3
[0.017s][info]    [gc,init] Concurrent Refinement Workers: 10
[0.017s][info]    [gc,init] Periodic GC: Disabled
[0.027s][info]    [gc,metaspace] CDS archive(s) mapped at: [0x000002a7ce000000-0x000002a7cebd0000-0x000
002a7cebd0000), size 12386304, SharedBaseAddress: 0x000002a7ce000000, ArchiveRelocationMode: 1.
[0.027s][info]    [gc,metaspace] Compressed class space mapped at: 0x000002a7cf000000-0x000002a80f0000
0, reserved size: 1073741824
[0.027s][info]    [gc,metaspace] Narrow klass base: 0x000002a7ce000000, Narrow klass shift: 0, Narrow k
lass range: 0x100000000
[0.078s][info]    [gc,start] GC(0) Pause Young (Normal) (G1 Evacuation Pause)
[0.078s][info]    [gc,task] GC(0) Using 2 workers of 10 for evacuation
[0.087s][info]    [gc,phases] GC(0) Pre Evacuate Collection Set: 0.1ms
[0.087s][info]    [gc,phases] GC(0) Merge Heap Roots: 0.1ms
[0.087s][info]    [gc,phases] GC(0) Evacuate Collection Set: 7.0ms
[0.088s][info]    [gc,phases] GC(0) Post Evacuate Collection Set: 0.3ms
[0.088s][info]    [gc,phases] GC(0) Other: 1.4ms
[0.088s][info]    [gc,heap] GC(0) Eden regions: 23->0(9)
[0.088s][info]    [gc,heap] GC(0) Survivor regions: 0->3(3)
[0.088s][info]    [gc,heap] GC(0) Old regions: 0->20
[0.089s][info]    [gc,heap] GC(0) Archive regions: 0->0
[0.089s][info]    [gc,heap] GC(0) Humongous regions: 0->0
[0.089s][info]    [gc,metaspace] GC(0) Metaspace: 176K(384K)->176K(384K) NonClass: 165K(256K)->165K(256
```

Explanation:

- `-Xmx64m` sets the JVM maximum heap size to 64MB, making memory fill up faster
- This triggers GC more frequently and aggressively
- You'll observe more GC events and potentially different GC behavior

4. Modern GC Logging (JDK 9+)

Try using `-Xlog:gc*` instead of `-XX:+PrintGCDetails` if your JDK ≥ 9 , otherwise you'll see this warning:
`[warning][gc] -XX:+PrintGCDetails is deprecated. Will use -Xlog:gc* instead.`

```
javac Example2.java
java -Xlog:gc* Example2
java -Xmx64m -Xlog:gc* Example2
```

Summary

This tutorial covered:

1. **Java Compilation and Execution:** From source code to bytecode to execution
2. **Class Loading:** Understanding what happens before your `main()` method runs
3. **Bytecode Analysis:** Examining the intermediate representation of Java code
4. **Memory Management:** Observing how the JVM handles memory allocation and garbage collection
5. **Performance Analysis:** Using GC logs to understand application behavior under different memory constraints

These skills form the foundation for understanding JVM internals, performance tuning, and debugging complex Java applications.